Interpretable Machine Learning - Assignment

Markus (nxm974), Patrick (gpk712), Jan (wrg291) & Rasmus (dsz631)

2026-06-04

Contents

Introduction **1**

Preprocessing **2**

Data **3**

Candidate models **5**
- Constant-prevalence baseline . . . 6
- Logistic regression . . . 6
- Penalized variants: Ridge, Lasso, Elastic Net . . . 7
- Generalized Additive Model . . . 9
- k-Nearest Neighbors . . . 11
- Tree-based learners . . . 12
- Random Forests . . . 13
- Neural Networks . . . 15
- Extreme Gradient Boosting (XGBoost) . . . 17

Choice of final model **19**

Interpretability of the final model **20**
- Global feature importance . . . 20
- Feature effects: PDP and ICE . . . 21
- Feature effects under correlation: ALE . . . 21
- Explanations with SHAP . . . 23
- Functional decomposition with glex . . . 24
- Interpretability summary . . . 27
- Sensitive and Protected Features . . . 28

Conclusion **30**

Appendix **30**

Introduction

We analyze the CDC Diabetes Health Indicators dataset to predict whether a patient has diabetes or pre-diabetes from their recorded characteristics. We treat this as a binary classification problem and compare parametric, non-parametric, and tree-based models as well as neural networks.

The dataset contains 253,680 patient records and 22 variables. The target, `Diabetes_binary`, is coded 0 for healthy patients and 1 for patients with pre-diabetes or diabetes. The remaining 21 predictors fall into four groups: demographic and socioeconomic, clinical, lifestyle, and healthcare access, summarized in Table 1.

Table 1: The 21 predictor variables, grouped by type.

Variable	Type	Description
Demographic and socioeconomic		
`Sex`	Binary	Sex (0 = female, 1 = male)
`Age`	Ordinal (1-13)	Age in 13 five-year brackets (18-24 to 80+)
`Education`	Ordinal (1-6)	Highest education level attained
`Income`	Ordinal (1-8)	Household income bracket
Clinical		
`BMI`	Continuous	Body mass index
`GenHlth`	Ordinal (1-5)	Self-rated general health (1 = excellent, 5 = poor)
`MentHlth`	Count (0-30)	Poor mental-health days in the past month
`PhysHlth`	Count (0-30)	Poor physical-health days in the past month
`HighBP`	Binary	High blood pressure
`HighChol`	Binary	High cholesterol
`CholCheck`	Binary	Cholesterol check within the last five years
`Stroke`	Binary	History of stroke
`HeartDiseaseorAttack`	Binary	Coronary heart disease or heart attack
`DiffWalk`	Binary	Serious difficulty walking or climbing stairs
Lifestyle		
`Smoker`	Binary	Smoked at least 100 cigarettes in lifetime
`HvyAlcoholConsump`	Binary	Heavy alcohol consumption
`PhysActivity`	Binary	Physical activity outside work in the past month
`Fruits`	Binary	Eats fruit at least once per day
`Veggies`	Binary	Eats vegetables at least once per day
Healthcare access		
`AnyHealthcare`	Binary	Has any health-insurance coverage
`NoDocbcCost`	Binary	Cost prevented a doctor visit in the past year

Preprocessing

```
raw_data <- read_csv("diabetes.csv")
set.seed(100)

clean_data <- raw_data %>%
  mutate(across(c(Diabetes_binary, HighBP, HighChol, CholCheck, Smoker,
                  Stroke, HeartDiseaseorAttack, PhysActivity, Fruits, Veggies,
                  HvyAlcoholConsump, AnyHealthcare, NoDocbcCost, DiffWalk, Sex),
                as.factor))


n <- nrow(clean_data)

pos_idx <- which(clean_data$Diabetes_binary == "1")
neg_idx <- which(clean_data$Diabetes_binary == "0")
train_idx <- sort(c(
  sample(pos_idx, size = floor(0.8 * length(pos_idx))),
  sample(neg_idx, size = floor(0.8 * length(neg_idx)))
))
```

```
test_idx <- setdiff(seq_len(n), train_idx)

train_data <- clean_data[train_idx, ]
test_data  <- clean_data[test_idx, ]

task      <- as_task_classif(train_data, target = "Diabetes_binary", positive = "1")
task_test <- as_task_classif(test_data,  target = "Diabetes_binary", positive = "1")

measures <- msrs(c("classif.logloss"))
cache_rds <- function(name, expr) {
  path <- file.path("model_cache", paste0(make.names(name), ".rds"))
  if (file.exists(path)) return(readRDS(path))
  obj <- expr
  saveRDS(obj, path)
  obj
}
leaderboard <- new.env()
leaderboard$tbl <- tibble::tibble(Model = character(), `Log-loss` = numeric())

add_result <- function(model_name, score) {
  if (is.null(leaderboard$tbl))
    leaderboard$tbl <- tibble::tibble(Model = character(), `Log-loss` = numeric())
  leaderboard$tbl <- dplyr::bind_rows(
    leaderboard$tbl,
    tibble::tibble(Model = model_name, `Log-loss` = round(as.numeric(score), 4))
  )
  invisible(leaderboard$tbl)
}
evaluate_on_test <- function(learner, model_name,
                             train_task = task, eval_task = task_test) {
  score <- cache_rds(paste0("score__", model_name), {
    learner$train(train_task)
    as.numeric(learner$predict(eval_task)$score(measures))
  })
  add_result(model_name, score)
  invisible(learner)
}
```

Data

The dataset has no missing values. We keep duplicate rows, since there is no ID column and they are likely separate patients with identical values. We convert the binary features and the target to factors. We keep the ordinal variables such as Age and GenHlth as integers. This preserves their order and lets the tree-based learners split on single thresholds instead of losing structure to one-hot encoding. It forces a linear-in-log-odds assumption on the logistic regression baseline, which we accept because the GAM below relaxes it where it matters.

Continuous features stay on their native scales, and standardization is left to the learners that need it. glmnet standardizes internally, and we scale by hand only for distance-based learners such as k-NN. We use an 80/20 stratified train and test split to keep the 14% target prevalence and to avoid selection optimism. The 80% training set with $n = 202{,}944$ is used for all fitting and internal cross-validation. The 20% holdout set with $n = 50{,}736$ is kept for evaluation and reported in one table that grows as each model is added.

We evaluate every model using log-loss, defined per observation as

$$L(y, \hat{p}) = -y \log \hat{p} - (1 - y) \log(1 - \hat{p}).$$

Log-loss rewards calibrated probabilities, not only correct classifications. Classification error is a poor fit here. At a prevalence of 14% the constant predictor $\hat{p} \equiv 0$ already reaches 86% accuracy without learning anything, and accuracy ignores how confident a wrong prediction was. Log-loss also handles class imbalance, since each observation contributes on the log scale of its own predicted probability, so the minority class is not drowned out. Thus we use log-loss as our metric throughout.

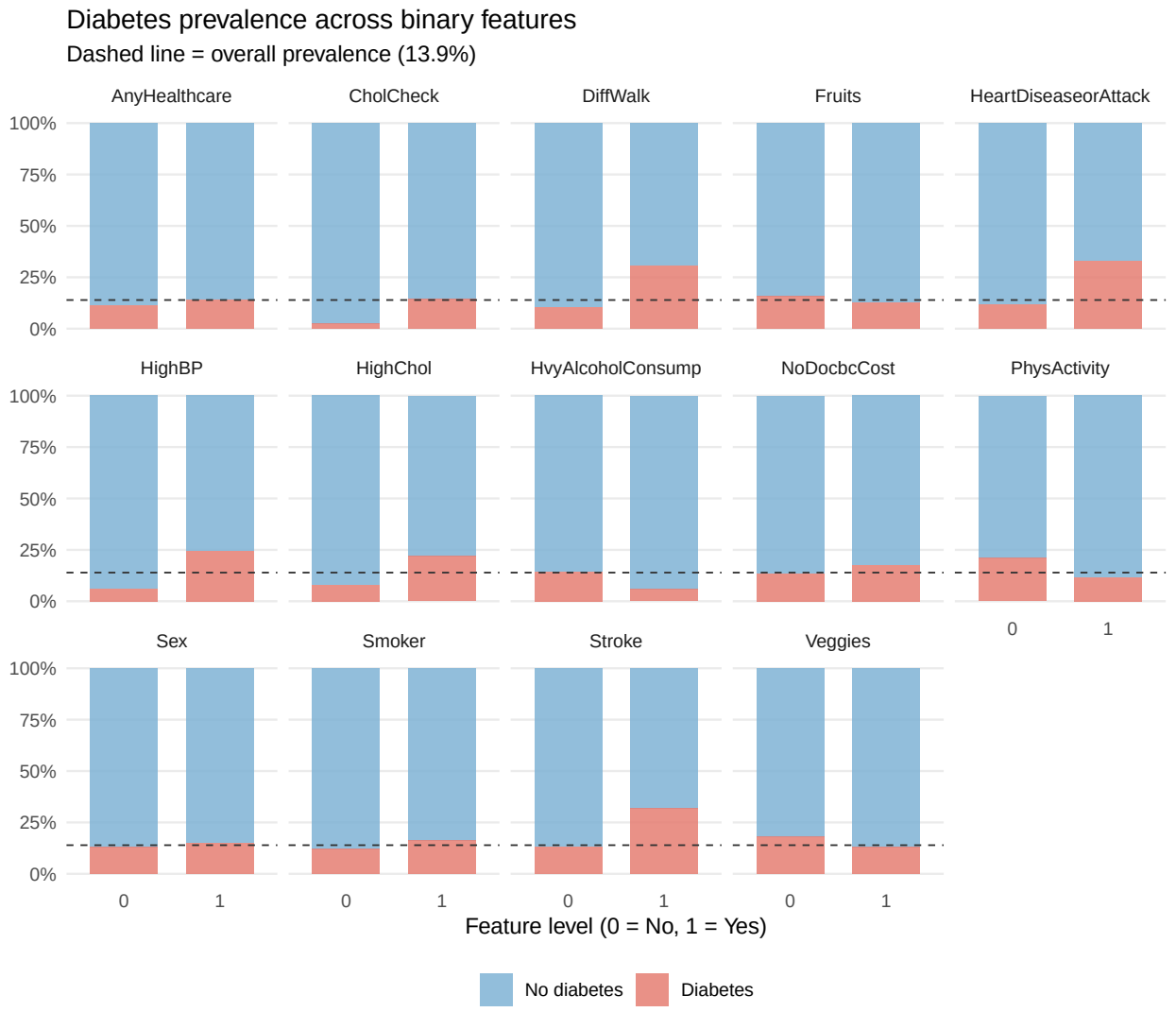


Figure 1: Diabetes prevalence across binary features. Bars show the share of each level that has/does not have diabetes. The dashed line marks overall prevalence (13.9%). Heights above the dashed line indicate features positively associated with diabetes.

Figure 1 shows diabetes prevalence across the binary features. The dashed line marks the overall 13.9% prevalence, so the height of the red segment in each bar relative to that line gives the direction of the association. A few patterns stand out. HighBP, HighChol, HeartDiseaseorAttack, Stroke and DiffWalk all about double the diabetes rate when present. CholCheck runs the other way. Respondents with no recent cholesterol check have near-zero prevalence, which reflects selection rather than protection, since people who

skip screening are also less likely to be diagnosed. HvyAlcoholConsump shows the same inverse pattern, likely from reverse causality, as diagnosed diabetics are told to cut alcohol. The lifestyle features Fruits, Veggies, PhysActivity and Smoker and the access features AnyHealthcare and NoDocbcCost separate the classes only a little, so they will contribute less on their own but may still matter in combination. Sex shows a tiny marginal effect, which we flag here because it returns in the sensitive-features section.

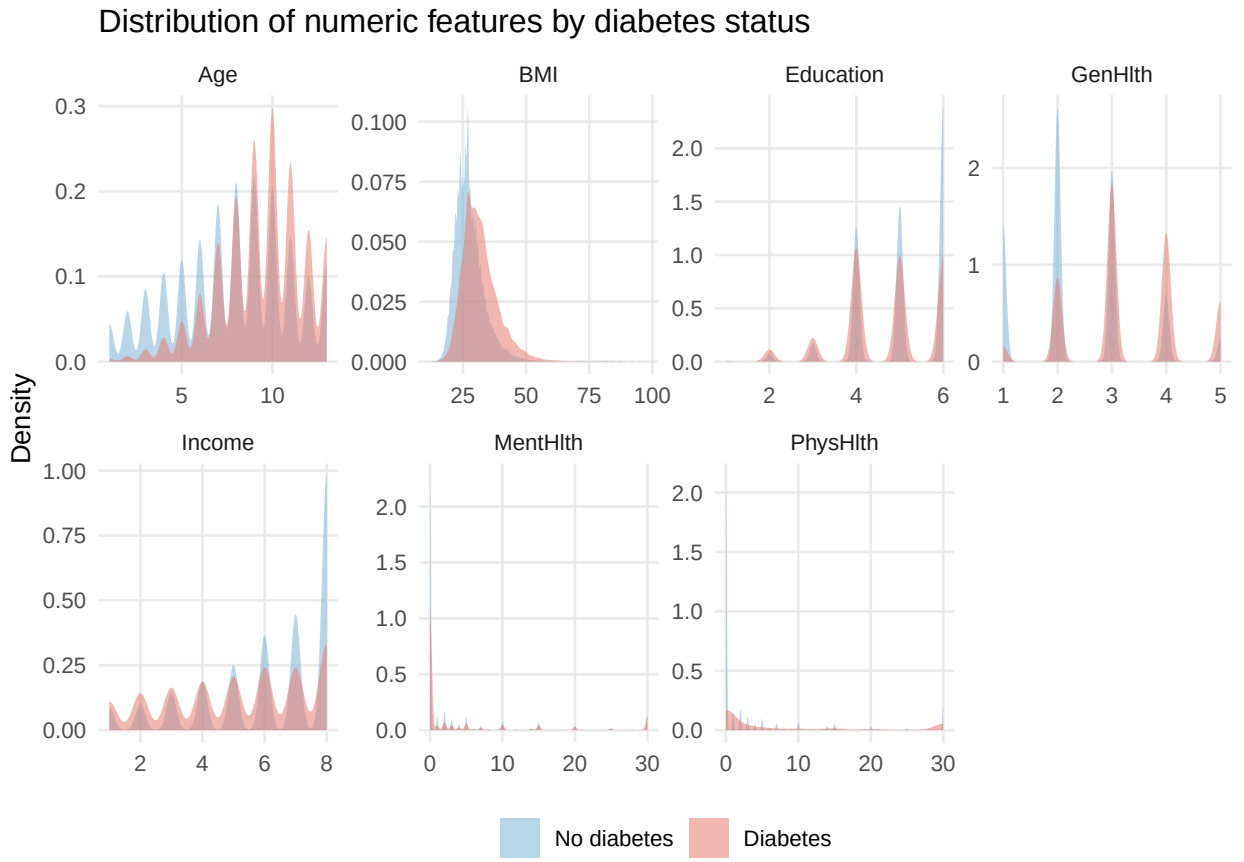


Figure 2: Distribution of the numeric features by diabetes status. Densities are shown separately for patients without diabetes (blue) and with diabetes or pre-diabetes (red), and the spikes on integer-valued features such as Age and GenHlth are artefacts of kernel smoothing between discrete levels.

We use density plots in figure 2 throughout for visual consistency. Many variables are integer-valued, so the kernel density estimator adds spikes when it interpolates between levels. Age shows the clearest signal, with the diabetic distribution shifted right toward 60 to 69 years. The diabetic group also has a higher median and heavier right tail for BMI, worse self-rated health, and more reported bad physical and mental health days. Education and Income show the expected gradients, though small ones. Overall the strongest predictors look like Age, GenHlth, BMI, HighBP, HighChol and cardiovascular history.

Candidate models

Under empirical risk minimization, choosing a learner means choosing $\hat{m} \in \arg\min_{m \in \mathcal{G}} \hat{R}_n(m)$, where $\hat{R}_n(m) = \frac{1}{n} \sum_{i=1}^{n} L(Y_i, m(X_i))$. The model class $\mathcal{G}$ encodes the assumptions we are willing to make. We introduce each candidate in turn and score it on the 20% held-out test set, adding the result to a running table so the leaderboard grows as the report goes on.

Constant-prevalence baseline

Before using any predictor, we fit a featureless classifier that predicts the marginal class proportions for every observation. Under log-loss this gives

$$L_0 = -\bar{y}\log\bar{y} - (1-\bar{y})\log(1-\bar{y}) \approx 0.404,$$

which is the smallest log-loss any predictor that ignores X can reach.

Table 2: Running test-set log-loss after the constant-prevalence baseline.

Model	Log-loss
Constant prevalence	0.4038

Logistic regression

Logistic regression uses the log-loss above and restricts the search space to the model class

$$\mathcal{G} = \left\{ x \mapsto \sigma(\beta_0 + x^\top \beta) \;\mid\; \beta_0 \in \mathbb{R},\ \beta \in \mathbb{R}^p \right\},$$

where $\sigma(z) = 1/(1+e^{-z})$. The resulting log-loss and fitted coefficients can be seen in table 3 and table 4 respectively.

Table 3: Running test-set log-loss after adding logistic regression.

Model	Log-loss
Constant prevalence	0.4038
Logistic Regression	0.3182

The drop from ≈ 0.403 to ≈ 0.318 shows that even a linear-in-log-odds model captures most of the signal. The demographic, clinical and behavioral variables together explain much of the structure beyond the marginal prevalence.

Table 4: Logistic-regression log odds (95% Wald CIs). A log odds > 0 raises the likelihood of diabetes, while a log odds < 0 lowers it. Values represent the additive change in the log-odds of the outcome per unit change in the predictor.

Variable	Log Odds	95% CI
CholCheck	1.31	1.16 to 1.47
HighBP	0.76	0.73 to 0.80
HighChol	0.58	0.55 to 0.61
GenHlth	0.53	0.51 to 0.55
Sex	0.26	0.23 to 0.29
HeartDiseaseorAttack	0.21	0.17 to 0.25
Age	0.12	0.12 to 0.13
Stroke	0.12	0.06 to 0.17
DiffWalk	0.12	0.08 to 0.15
AnyHealthcare	0.08	0.01 to 0.16
BMI	0.06	0.06 to 0.06
NoDocbcCost	0.02	-0.03 to 0.07
MentHlth	0.00	-0.01 to -0.00
PhysHlth	-0.01	-0.01 to -0.01
Smoker	-0.01	-0.04 to 0.02
Veggies	-0.02	-0.05 to 0.02
Education	-0.03	-0.04 to -0.01
Income	-0.05	-0.06 to -0.04
Fruits	-0.05	-0.08 to -0.02
PhysActivity	-0.06	-0.09 to -0.02
HvyAlcoholConsump	-0.78	-0.87 to -0.70

Each coefficient is the additive effect on the log-odds of diabetes, holding the other features fixed. HighBP, HighChol and a history of heart disease or stroke each raise the odds, in line with the EDA. CholCheck and HvyAlcoholConsump sit on the same side as in the prevalence plot, and we read both as reverse causality rather than effects. Undiagnosed people skip cholesterol screening, and diagnosed diabetics are told to cut alcohol. For the ordinals kept numeric, the coefficient is per one-level step and only means something if the effect is linear on the log-odds scale, which is the assumption the GAM smooths examine next.

Penalized variants: Ridge, Lasso, Elastic Net

We next add three penalized variants of the same logistic model. Ridge uses the ℓ_2 penalty $J_\lambda(\beta) = \lambda\|\beta\|_2^2$, which shrinks every coordinate. Lasso uses the ℓ_1 penalty $J_\lambda(\beta) = \lambda\|\beta\|_1$, which selects features by driving some coefficients to zero. Elastic Net sits between the two and suits correlated groups such as the cardiovascular variables HighBP, HighChol, HeartDiseaseorAttack and Stroke. For each model, λ is chosen by 5-fold inner cross-validation with `cv.glmnet` and `type.measure = "deviance"`, so the inner objective matches our outer log-loss metric. We report the Lasso at `lambda.min`, the best CV deviance, and at `lambda.1se`, the largest penalty within one standard error of the minimum, which gives a sparser and more readable model. The results of these models are reported in table 5.

```
# Treatment encoding drops a reference category and represents each factor with
# n-1 dummy columns, needed for GLMNet.
encode <- po("encode", method = "treatment")

make_glmnet <- function(alpha, s) {
  as_learner(encode %>>% lrn("classif.cv_glmnet",
                             predict_type = "prob",
                             alpha        = alpha,
```

```
                              type.measure = "deviance",
                              nfolds       = 5,
                              s            = s))
}

lrn_ridge     <- make_glmnet(alpha = 0,   s = "lambda.min")
lrn_lasso_min <- make_glmnet(alpha = 1,   s = "lambda.min")
lrn_lasso_1se <- make_glmnet(alpha = 1,   s = "lambda.1se")
lrn_enet      <- make_glmnet(alpha = 0.5, s = "lambda.min")

lrn_ridge     <- evaluate_on_test(lrn_ridge,     "Ridge")
lrn_lasso_min <- evaluate_on_test(lrn_lasso_min, "Lasso (lambda.min)")
lrn_lasso_1se <- evaluate_on_test(lrn_lasso_1se, "Lasso (lambda.1se)")
lrn_enet      <- evaluate_on_test(lrn_enet,      "Elastic Net (alpha=0.5)")
```

Table 5: Running test-set log-loss after adding the penalized variants.

Model	Log-loss
Constant prevalence	0.4038
Logistic Regression	0.3182
Ridge	0.3187
Lasso (lambda.min)	0.3182
Lasso (lambda.1se)	0.3191
Elastic Net (alpha=0.5)	0.3182

The unpenalized logistic regression and the three glmnet variants give almost identical log-loss. This is what Chapter 3 predicts. The conditional excess risk of ordinary least squares is $\frac{\sigma^2}{n}\,\mathrm{tr}(\Sigma\hat{\Sigma}^{-1}) \approx \sigma^2 p/n$ by Theorem 3.2.2. With $p = 21$ and $n \approx 203{,}000$ that variance term is of order 10^{-4}, so there is almost nothing for shrinkage to remove and any penalty can only add bias. Theorem 3.2.2 is a squared-loss OLS result, and we use it here as the analogue for logistic regression, whose maximum-likelihood estimator has an asymptotic variance that scales the same way as p/n through the Fisher information. At this sample size regularization buys interpretability, not accuracy.

Inspecting Lasso sparsity

To see which features the Lasso treats as redundant, we extract its coefficients at `lambda.min` and `lambda.1se`. These are reported in table 6. We treat `lambda.1se` as our interpretable baseline, trading a little fit for a sparser and more readable model.

Table 6: Lasso coefficients at lambda.min and lambda.1se. Variables marked yes are shrunk to exactly zero.

Variable	Lasso (lambda.min)	Lasso (lambda.1se)	Dropped at lambda.min	Dropped at lambda.1se
AnyHealthcare	0.057	0.000		yes
Veggies	-0.013	0.000		yes
PhysHlth	-0.006	0.000		yes
MentHlth	-0.004	-0.001		yes
NoDocbcCost	0.003	0.000		yes
Smoker	-0.001	0.000		yes
CholCheck	1.220	0.743		
HighBP	0.760	0.745		
HvyAlcoholConsump	-0.750	-0.541		
HighChol	0.569	0.530		
GenHlth	0.526	0.488		
Sex	0.248	0.180		
HeartDiseaseorAttack	0.208	0.204		
Age	0.122	0.113		
DiffWalk	0.110	0.071		
Stroke	0.109	0.057		
BMI	0.060	0.057		
Income	-0.051	-0.042		
PhysActivity	-0.050	-0.014		
Fruits	-0.048	-0.003		
Education	-0.028	-0.020		

At `lambda.1se` the Lasso keeps 15 of the 21 predictors. It keeps the main clinical variables CholCheck, HighBP, HvyAlcoholConsump, HighChol and GenHlth, along with Sex, HeartDiseaseorAttack, Age, DiffWalk, BMI, Income and Education. It drops the variables that looked weak in the EDA, namely Smoker, NoDocbcCost, AnyHealthcare, Veggies, MentHlth and PhysHlth, which confirms they add no signal once the main clinical features are present. The small shrinkage between `lambda.min` and `lambda.1se` for the strong predictors shows the bias-variance trade-off. The Lasso takes a small downward bias to cut variance, shrinking stable signals a little and collapsing weak ones to zero.

Generalized Additive Model

To relax the linear-in-log-odds assumption without paying the full curse of dimensionality, we fit a Generalized Additive Model with regression function

$$m^*(x) = m_0 + \sum_{j=1}^{p} m_j^*(x_j).$$

Chapter 5 shows that additivity bypasses the curse of dimensionality (the $n^{-4/(4+p)}$ rate in $\mathbb{R}^p$), allowing twice continuously differentiable components m_j^* to be estimated at the one-dimensional nonparametric rate of $n^{-4/5}$. Because a full smoothing spline with knots at all $n \approx 200{,}000$ observations is computationally infeasible, we use penalized regression splines, which is a reduced-rank basis regularized by a roughness penalty like $\lambda \int (m''(x))^2\, dx$. We smooth all sufficiently granular numeric predictors (BMI, Age, GenHlth, MentHlth, PhysHlth, Income, and Education). Rather than assuming linearity a priori, we let the roughness penalty determine the shape. Because an unnecessary smooth collapses to roughly one effective degree of freedom, this flexibility costs almost no variance. The partial-effect plots below confirm this, as the penalty successfully recovers near-linear effects for GenHlth and Income, and flat effects for MentHlth and PhysHlth. We cap the basis dimension k at one below the number of distinct values for ordinals, use mgcv defaults for continuous/count predictors, and model binary variables linearly. The log-loss of the fitted GAM model is reported in table 7.

```
gam_formula <- Diabetes_binary ~ s(BMI) + s(Age, k = 12) + s(GenHlth, k = 4) +
  s(MentHlth) + s(PhysHlth) + s(Income, k = 7) + s(Education, k = 5) +
  HighBP + HighChol + CholCheck + Smoker + Stroke + HeartDiseaseorAttack +
  PhysActivity + Fruits + Veggies + HvyAlcoholConsump + AnyHealthcare +
  NoDocbcCost + DiffWalk + Sex

lrn_gam <- lrn("classif.gam", predict_type = "prob", formula = gam_formula)
lrn_gam <- evaluate_on_test(lrn_gam, "GAM (Splines)")
```

Table 7: Running test-set log-loss after adding the GAM.

Model	Log-loss
Constant prevalence	0.4038
Logistic Regression	0.3182
Ridge	0.3187
Lasso (lambda.min)	0.3182
Lasso (lambda.1se)	0.3191
Elastic Net (alpha=0.5)	0.3182
GAM (Splines)	0.3128

The GAM gives a small but consistent improvement over the parametric models.

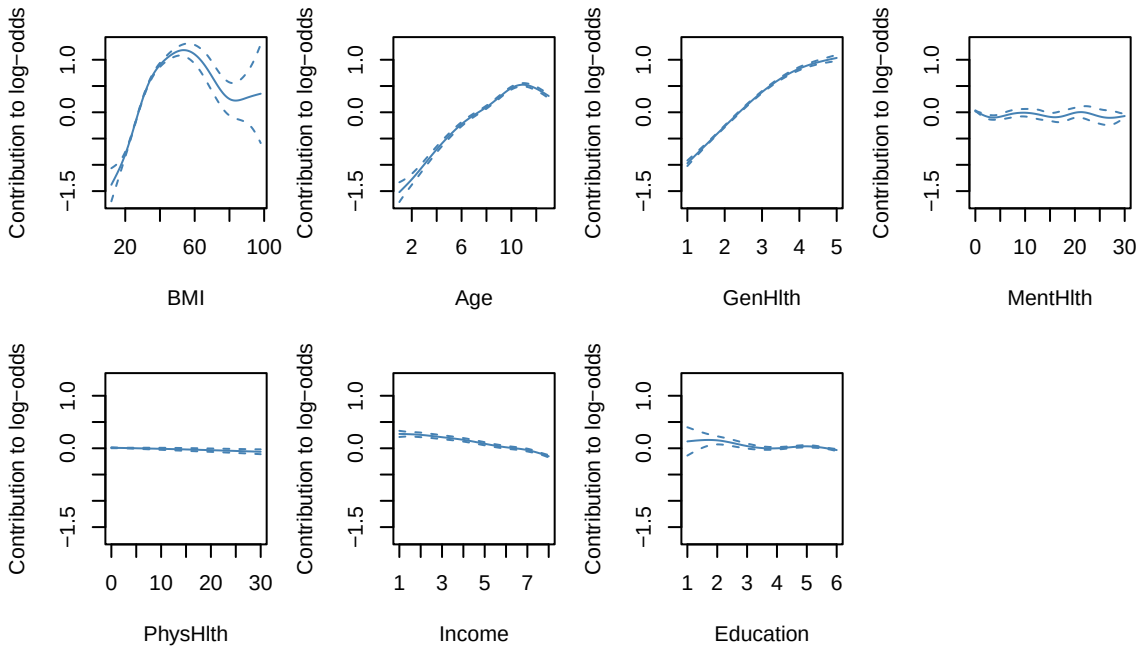


Figure 3: Estimated GAM partial effects for the seven smoothed predictors, on the log-odds scale. Each panel shows the centred contribution of one feature with a 95% confidence band

To see what the GAM captures, we refit it with `mgcv::gam` and plot the partial effect of each smoothed predictor on the log-odds. The result can be seen in figure 3.

The seven smooths show that most of the nonlinearity sits in BMI and Age. BMI is non-monotone, rising to a peak near BMI 50 and then declining, while Age rises before flattening and dipping a little in the oldest

groups. GenHlth and Income are close to linear, with GenHlth increasing and Income decreasing. MentHlth and PhysHlth are flat across their ranges, which confirms the Lasso result that they add little once the other variables are present. So the GAM gains over logistic regression mainly through the nonlinear effects of BMI and Age, and a logistic model with flexible terms for those two would likely capture most of the gain while staying interpretable. If trees beat the GAM, interactions are the likely reason.

k-Nearest Neighbors

We include k-nearest-neighbors as a non-parametric baseline. Theorem 5.1.3 is stated for squared-loss regression with regression function $m^*(x) = \mathbb{E}[Y \mid X = x]$. For binary Y this is the conditional class probability $\mathbb{P}(Y = 1 \mid X = x)$, which is what we estimate. The theorem gives a prediction-error rate of order $n^{-2/(2+p)}$ reached by

$$k \approx n^{2/(2+p)},$$

which for our training sample with $n \approx 203{,}000$ and $p = 21$ gives $k \approx 3$.

This k minimizes MSE under Lipschitz continuity, but log-loss treats errors very differently from squared loss. A wrong predicted probability ε costs $-\log \varepsilon$ rather than at most 1, and this is worst at small k. With $k = 3$ and uniform weights a predicted probability can only take the values $\{0, 1/3, 2/3, 1\}$, so a neighborhood that votes the wrong way unanimously gives a confident wrong probability and inflates the log-loss. The default kknn kernel down-weights distant neighbors and smooths these values, but with three neighbors they stay coarse and near the extremes, so the point holds. The theorem's k is the wrong target when calibration matters, so we use the $k \approx \sqrt{n}$ rule of thumb, where averaging over many neighbors gives smooth probabilities. The Lipschitz assumption is also doubtful here, since our predictors mix binary indicators, ordinal integers and continuous variables, and Euclidean Lipschitz continuity only constrains m^* at the $\{0, 1\}^p$ corners of the binary subspace. We treat both k choices as heuristics rather than direct uses of the theorem. k-NN needs a numeric feature space on a common scale, so its preprocessing differs from the other learners. We treatment-encode the factors. Every factor here is binary, so this gives one 0/1 indicator per variable and loses no distance information relative to one-hot encoding. We then standardize the continuous and ordinal predictors BMI, Age, GenHlth, MentHlth, PhysHlth, Income and Education to unit variance, so a wide-range feature such as BMI does not dominate the distance by scale alone. The result of the fit is reported in table 8.

```
set.seed(100)
k_practical <- round(sqrt(task$nrow))

scale_numeric <- po("scale", affect_columns = selector_type(c("numeric", "integer")))
lrn_knn <- as_learner(
  encode %>>% scale_numeric %>>%
    lrn("classif.kknn", predict_type = "prob", k = k_practical)
)
lrn_knn <- evaluate_on_test(lrn_knn, "KNN")
```

Table 8: Running test-set log-loss after adding the KNN

Model	Log-loss
Constant prevalence	0.4038
Logistic Regression	0.3182
Ridge	0.3187
Lasso (lambda.min)	0.3182
Lasso (lambda.1se)	0.3191
Elastic Net (alpha=0.5)	0.3182
GAM (Splines)	0.3128
KNN	0.3205

KNN reaches a log-loss of 0.3205, close to the parametric models but marginally worse than both logistic regression and the GAM. Averaging over the $k \approx \sqrt{n} \approx 450$ nearest neighbours keeps the predicted probabilities smooth enough to stay calibrated, which is why it avoids the blow-up that the small-k MSE-optimal rule would have caused under log-loss. It still trails the parametric and additive fits, since a single Euclidean metric weighs every standardized feature equally and cannot prioritise the strong predictors the way a fitted model can.

Tree-based learners

We now shift our focus to decision trees and tree-based learners. Decision trees are a natural candidate for classification tasks due to their interpretability and ability to capture interaction effects without explicit feature engineering. However, single trees are known to be high-variance learners, since small changes in the training data can lead to very different tree structures. This motivates investigating both pruning strategies and ensemble methods to improve generalization performance. First we start by fitting a single tree of depth 30 to the dataset, without considering any form of pruning.

```
full_tree_learner <- lrn("classif.rpart", predict_type = "prob", cp = 0)
full_tree <- evaluate_on_test(full_tree_learner, model_name = "Depth 30 Tree")
```

```
set.seed(100)
at_pruning = auto_tuner(
  tuner = mlr3tuning::tnr("grid_search", resolution = 100, batch_size = 16),

  learner = lrn("classif.rpart", predict_type = "prob"),

  resampling = rsmp("cv", folds = 5),

  measure = msr("classif.logloss"),

  search_space = ps(
    cp = p_dbl(1e-5, 1e-1, logscale = TRUE)
  ),

  terminator = mlr3tuning::trm("none")
)

pruned_tree <- evaluate_on_test(at_pruning, "Pruned Tree")
```

Table 9: Running test-set log-loss after deep tree and pruned tree.

Model	Log-loss
Constant prevalence	0.4038
Logistic Regression	0.3182
Ridge	0.3187
Lasso (lambda.min)	0.3182
Lasso (lambda.1se)	0.3191
Elastic Net (alpha=0.5)	0.3182
GAM (Splines)	0.3128
KNN	0.3205
Depth 30 Tree	0.4026
Pruned Tree	0.3495

As can be seen in Table 9 the deep tree performs very poorly, getting a log-loss on the test set that is almost

at the level of the featureless model. This is likely due to the extreme depth of the tree allowing for overfitting to the training data which in turn leads to poor performance on new unseen data. This is consistent with the bias-variance trade-off: an unregularized deep tree has very low bias but high variance, leading to poor generalization despite strong training performance.

To reduce variance and improve generalization, we introduce regularization through cost-complexity pruning. This discourages overly complex trees by penalizing the number of terminal nodes, effectively trading a small increase in bias for a hopefully large reduction in variance. We then tune this penalty parameter to get the best performing tree. In mlr3 this pruning is done through tuning the cp hyperparameter. Doing this resulted in a cp parameter of 0.0001023531.

As seen in Table 9 this significantly improves the performance of the model, which is not surprising as the pruned tree has cut the number of leaves down from 3225 to 361, allowing for better generalization performance.

Random Forests

While pruning reduces variance in a single tree, it does not fully address the instability inherent in tree-based learners. Random forests mitigate this issue by aggregating many decorrelated trees trained on bootstrap samples of the data and random subsets of features. This ensemble approach reduces variance further while maintaining the expressive power of decision trees. It does however come at the cost of a massive hit in interpretability, as it may be easy to interpret a single tree but understanding the interactions in 500 different trees is much more challenging. We commence by fitting a random forest with the default parameters of mlr3. That is an ensemble of 500 trees, with an mtry of $\sqrt{p}$ a minimum node size for split of 1 and no maximum tree depth.

```
set.seed(100)
default_forest_learner <- lrn("classif.ranger", predict_type = "prob")
default_forest <- evaluate_on_test(default_forest_learner, "Default Random Forest")
```

Table 10: Running test-set log-loss after fitting of random forests.

Model	Log-loss
Constant prevalence	0.4038
Logistic Regression	0.3182
Ridge	0.3187
Lasso (lambda.min)	0.3182
Lasso (lambda.1se)	0.3191
Elastic Net (alpha=0.5)	0.3182
GAM (Splines)	0.3128
KNN	0.3205
Depth 30 Tree	0.4026
Pruned Tree	0.3495
Default Random Forest	0.3149

As can be seen in Table 10, a random forest with the default parameters performs very well, compared to a lot of the other models discussed so far.

Although random forests are relatively robust to hyperparameter choices, performance can still be improved by controlling the strength of decorrelation between trees and the complexity of individual trees. Tuning the hyperparameters of the forest therefore aims to balance tree strength and diversity, which combined determine the performance of the ensemble. Thus we try to tune some of the many hyperparameters available for the random forest. Some of these include mtry, which is the number of features that are considered for

each split and sample.fraction which is the fraction of the total size of the dataset that is sampled when growing each tree (by default an entire bootstrap sample of size 202943 is generated from the dataset for each tree). These two parameters should impact the decorrelation between the trees. We also tune some of the split conditions, such as the minimum node members required to split (min.node.size) and the minimum amount of elements allowed in a terminal node (min.bucket). We also tune the max depth (max.depth) of the trees. These parameters should impact the complexity of the individual trees. Finally, we also tune the number of trees (num.trees). As fitting forests takes a long time we use the hyperband tuning procedure and define the number of trees as a fidelity parameter. The hyperband procedure tunes over a series of stages, where in the first stage many random choices of hyperparameters at a low amount of trees (low budget) are fitted and then the best performing candidates will move on to the next stage and be fitted on a larger amount of trees and this process continues until the maximum budget of 500 trees is reached. The best performing set of hyperparameters from the tuning can be seen in Table 11. The performance of the tuned modelled, as reported in table 12, is marginally better than the model using the default hyperparameters. The relatively small improvement in performance does however seem to confirm the statement that random forests are relatively robust to hyperparameter choices.

```
set.seed(100)
at_forest = auto_tuner(
  tuner = mlr3tuning::tnr("hyperband", eta = 4, repetitions = 1),
  learner = lrn("classif.ranger", predict_type = "prob"),
  resampling = rsmp("cv", folds = 5),
  measure = msr("classif.logloss"),

  search_space = ps(
    mtry = p_int(1, 10),
    sample.fraction = p_dbl(0.1, 1),
    min.node.size = p_int(1, 10),
    min.bucket = p_int(1, 10),
    num.trees = p_int(10, 500, tags = "budget"),
    max.depth = p_int(10, 30)
  ),
  terminator = mlr3tuning::trm("none")
)
tuned_forest <- evaluate_on_test(at_forest, "Tuned Random Forest")
```

Table 11: Tuned Hyperparameters for random forest

Hyperparameter	value
Mtry	6.0000000
Sample Fraction	0.4745814
Min Node Size	4.0000000
Min Bucket Size	3.0000000
Number of Trees	500.0000000
Max Depth	11.0000000

Table 12: Running test-set log-loss after fitting of tuned random forests.

Model	Log-loss
Constant prevalence	0.4038
Logistic Regression	0.3182
Ridge	0.3187
Lasso (lambda.min)	0.3182
Lasso (lambda.1se)	0.3191
Elastic Net (alpha=0.5)	0.3182
GAM (Splines)	0.3128
KNN	0.3205
Depth 30 Tree	0.4026
Pruned Tree	0.3495
Default Random Forest	0.3149
Tuned Random Forest	0.3122

Neural Networks

As another type of learner, we consider neural networks, specifically multi-layer perceptron (MLP) models. Neural networks represent a fundamentally different class of models compared to the other learners discussed in this report. While trees partition the feature space using axis-aligned splits, multilayer perceptrons learn continuous nonlinear transformations of the input space. This makes them particularly powerful for capturing smooth and high-dimensional decision boundaries, at the cost of increased tuning complexity and reduced interpretability. As this is the only black-box predictor introduced in this course other than the tree-based predictors, it is natural to see if this different modelling approach can improve upon the trees. To begin with we try fitting a simple neural network with two hidden layers both with 50 neurons. We train it over 100 epochs.

```
mlp = as_learner(po("encode", method = "treatment") %>>% lrn("classif.mlp",
  neurons = c(50, 50),
  batch_size = 4096,
  epochs = 100,
  device = "cuda",
  jit_trace = TRUE,
  predict_type = "prob",
  callbacks = t_clbk("progress"),
  seed = 100
))

untuned_mlp <- evaluate_on_test(mlp, "Untuned Neural Network")
```

The performance, seen in table 14, is slightly worse than the default random forest, but better than the models trained using regression methods (except for GAM). The performance of neural networks, however, is highly sensitive to choices of the model's architecture such as the number of hidden layers and neurons per hidden layer, as well as the different training parameters such as the number of epochs. Unlike tree-based models, there is no strong default configuration, thus we attempt to improve the performance of the neural network by tuning the number of hidden layers as well as the neurons in each layer. We also tune the number of epochs. The search space code for the neuron layers is taken from hyperparameter_optimization in the mlr3 book. Here we employ a successive halving tuner, with the fidelity parameter being the number of epochs. It works like hyperband tuning, but we can select the amount of configurations tested at the lowest budget. For this report we selected 100 initial candidates which made the tuning take about an hour on an Nvidia 5070ti GPU. The resulting tuned parameters, shown in Table 13, are practically identical to the

ones from the initial fit of the model and the difference in performance, as reported in table 14, is still not very large either and comes nowhere close to the performance of the tuned random forest, so the tuning did not have a very large effect. One could likely achieve better results by trying further configurations, as well as tuning other hyperparameters such as the learning rate and the non-linear transformations used in the network. However, training time is the enemy when tuning these neural networks.

```
set.seed(100)
at_mlp = auto_tuner(
  tuner = mlr3tuning::tnr("successive_halving", n = 100, eta = 3),
  learner = as_learner(po("encode", method = "treatment") %>>% lrn("classif.mlp",
  batch_size = 4096,
  device = "cuda",
  jit_trace = TRUE,
  predict_type = "prob",
  seed=100
)),
  resampling = rsmp("cv", folds = 5),
  measure = msr("classif.logloss"),
  # We need to prefix the parameters with classif.mlp since our base learner
  # is a combinition of po("encode") and classif.mlp
    search_space = ps(
      num_layers = p_int(lower = 1, upper = 20),
      num_neurons_per_layer = p_int(4, 64),
      .extra_trafo = function(x, param_set) {
        x$classif.mlp.neurons= rep(x$num_neurons_per_layer, x$num_layers)
        x$num_layers = NULL
        x$num_neurons_per_layer = NULL
        x
      },
      classif.mlp.epochs = p_int(30, 100, tags = "budget")
    ),
  terminator = mlr3tuning::trm("none")
)
tuned_mlp <- evaluate_on_test(at_mlp, "Tuned MLP")
```

Table 13: Tuned Hyperparameters for MLP

Hyperparameter	value
Number of hidden layers	2
Neurons per hidden layer	44
Number of epochs	90

Table 14: Running test-set log-loss after fitting of MLPs.

Model	Log-loss
Constant prevalence	0.4038
Logistic Regression	0.3182
Ridge	0.3187
Lasso (lambda.min)	0.3182
Lasso (lambda.1se)	0.3191
Elastic Net (alpha=0.5)	0.3182
GAM (Splines)	0.3128
KNN	0.3205
Depth 30 Tree	0.4026
Pruned Tree	0.3495
Default Random Forest	0.3149
Tuned Random Forest	0.3122
Untuned MLP	0.3178
Tuned MLP	0.3154

Extreme Gradient Boosting (XGBoost)

As the neural network was unable to improve upon the tuned random forest, we shift our focus back to tree based learners. Random forests reduce variance by averaging many decorrelated trees, but they do not directly reduce the bias of the individual trees. Boosting takes a different approach. By fitting trees sequentially, each new tree focuses on the mistakes of the current ensemble, allowing the model to gradually reduce approximation bias. XGBoost combines this idea with shrinkage and subsampling to control variance, making it a natural candidate to compare against the random forest. Unlike random forests however, boosted trees are generally sensitive to their hyperparameters. The learning rate, tree depth and number of boosting rounds jointly determine the bias-variance trade-off of the ensemble. A large learning rate or overly deep trees can fit the training data too aggressively and overfit, while a small learning rate or too few boosting rounds may leave substantial bias. Since there is no universally good default configuration, we decide to tune a few hyperparameters. The learning rate `eta` is one of these, and smaller values generalize better but need more boosting rounds `nrounds`, so the two must be tuned together. We control tree complexity with the maximum depth `max_depth` and with the row fraction `subsample` and column fraction `colsample_bytree` drawn for each tree. We choose the hyperparameters by random search over 30 configurations. The best performing set of hyperparameters can be seen in table 15 and the performance of the tuned model can be seen in table 16.

```
set.seed(100)
lrn_xgb <- lrn("classif.xgboost",
               eta              = to_tune(0.01, 0.3),
               nrounds          = to_tune(p_int(50, 500)),
               max_depth        = to_tune(p_int(3, 8)),
               subsample        = to_tune(0.5, 1),
               colsample_bytree = to_tune(0.5, 1),
               predict_type     = "prob")

graph_learner_xgb <- as_learner(
  po("encode",
     method = "treatment",
     affect_columns = selector_type(c("factor", "ordered"))) %>>%
    po("removeconstants") %>>%
    lrn_xgb
)
```

```r
at_xgb <- auto_tuner(
  tuner = mlr3tuning::tnr("random_search", batch_size = 10),
  learner    = graph_learner_xgb,
  resampling = rsmp("cv", folds = 5),
  measure    = msr("classif.logloss"),
  terminator = trm("evals", n_evals = 30)
)
```

```r
xgb_results <- cache_rds("xgb_results", {
  at_xgb$train(task)
  score <- as.numeric(at_xgb$predict(task_test)$score(measures))
  xgb_model  <- attr(at_xgb$learner$model$classif.xgboost$model[[1]], "model")
  #As this will be our final model we record the interpretability objects:
  importance <- xgb.importance(model = xgb_model)
  encode_po    <- po("encode", method = "treatment",
                     affect_columns = selector_type(c("factor", "ordered")))
  encoded_task <- encode_po$train(list(task))[[1]]
  train_df     <- as.data.frame(encoded_task$data())
  train_matrix <- as.matrix(train_df[, !names(train_df) %in% "Diabetes_binary"])
  shap_matrix  <- train_matrix[sample(nrow(train_matrix), 3000), ]
  shap         <- shapviz(xgb_model, X_pred = shap_matrix, X = shap_matrix)

  list(score = score, importance = importance, shap = shap, model = at_xgb)
})

at_xgb <- xgb_results$model

add_result("XGBoost (tuned)", xgb_results$score)
```

Table 15: Tuned Hyperparameters for XGBoost

Hyperparameter	value
Column Fraction	0.9928088
Eta (Learning Rate)	0.0407453
Max Depth	4.0000000
Number of Rounds	296.0000000
Row fraction	0.6512655

Table 16: Test-set log-loss across all candidate models, including tuned XGBoost.

Model	Log-loss
Constant prevalence	0.4038
Logistic Regression	0.3182
Ridge	0.3187
Lasso (lambda.min)	0.3182
Lasso (lambda.1se)	0.3191
Elastic Net (alpha=0.5)	0.3182
GAM (Splines)	0.3128
KNN	0.3205
Depth 30 Tree	0.4026
Pruned Tree	0.3495
Default Random Forest	0.3149
Tuned Random Forest	0.3122
Untuned MLP	0.3178
Tuned MLP	0.3154
XGBoost (tuned)	0.3098

Choice of final model

Our evaluation shows that the tuned XGBoost model achieves the lowest log-loss on the held-out test set, outperforming both the parametric baselines and the GAM. This predictive advantage stems from the ensemble's ability to natively learn nonlinearities and interaction structures directly from the data.

```
# Evaluate default 0.5 threshold
pred_xgb <- at_xgb$predict(task_test)
acc_05 <- pred_xgb$score(msr("classif.acc"))

prevalence_threshold <- mean(train_data$Diabetes_binary == "1")
pred_xgb$set_threshold(prevalence_threshold)
acc_prev <- pred_xgb$score(msr("classif.acc"))

nir <- max(prop.table(table(test_data$Diabetes_binary)))
```

Table 17: Performance of the tuned XGBoost on the held-out test set comparing the default 0.5 cut-off to the baseline prevalence threshold.

Metric	Value
Accuracy (0.5 threshold)	0.8678
Accuracy (prevalence threshold)	0.7229
No-information rate	0.8607

Table 18: Confusion matrix for the tuned XGBoost on the held-out test set using the baseline prevalence threshold. Rows are model predictions, columns the true diabetes status.

	True status	
Prediction	Diabetes (1)	No diabetes (0)
Diabetes (1)	5664	12651
No diabetes (0)	1406	31016

Accuracy tells a cautionary story regarding class imbalance. At the default 0.5 threshold, the model appears highly accurate but, as Table 17 shows, barely clears the no-information rate, masking the fact that it misses the vast majority of true diabetic cases by defaulting to the dominant healthy class. Shifting the decision threshold down to the baseline diabetes prevalence captures significantly more true positives, as the confusion matrix in Table 18 shows, though it trades away some overall accuracy to do so. While a metric like the Area Under the ROC Curve is traditionally used to evaluate threshold-independent performance under severe class imbalance, we deliberately restrict our methodology to the evaluation measures formally presented in this course. Consequently, we base our final model selection purely on log-loss and not classification accuracy. Our evaluation shows that the tuned XGBoost model achieves the lowest log-loss on the held-out test set, outperforming the GAM because its sequential boosting natively captures complex multidimensional interactions that our pre-specified smooth splines missed.

This is usually told as an accuracy-interpretability trade-off, but this is not always the case. The GAM recovers roughly 97% of XGBoost's improvement over the constant baseline at a log-loss cost of only 0.003, and its sole real deficit is the interaction structure an additive model omits by construction. Since those are dominated by a few pairwise terms, a GAM augmented with a handful of interaction terms would likely close most of the remaining gap while keeping its partial effects in closed form. Interpretability and accuracy are therefore not genuinely opposed here, but describe a frontier we can move along. For a clinical screening tool, where transparency carries importance, the GAM would be a defensible final choice. We nonetheless retain XGBoost, which attains the lowest log-loss with no hand-specified interactions, and devote the rest of this section to recovering its logic post hoc.

Interpretability of the final model

Most of these post hoc explanations run through the `iml` package, which wraps an mlr3 learner in a single `Predictor` object. We first use global methods, which describe the model on average, then local methods, which explain one prediction. We close with the functional decomposition, which is the common parent of these summaries and the only method here that isolates an interaction on its own.

Global feature importance

We rank features by permutation feature importance, which measures how much test-set performance degrades when a feature's values are shuffled, breaking its association with the target while preserving its marginal distribution. We use log-loss to match our evaluation metric and average over ten repetitions. Permutation importance is performance-based.

The bars in Figure 4 report the factor by which test-set log-loss worsens when a feature is shuffled, so a value of 1 marks a feature the model does not use and longer bars mark heavier reliance. The top features are GenHlth, BMI, Age and HighBP, followed by HighChol. The rest sit just above 1. This ordering matches the EDA and the GAM plots. The variables with the largest prevalence gaps and the only nonlinear smooths are the ones the model relies on, and the predictors the Lasso had already dropped sit at the bottom. Permutation importance is marginal, so each feature is shuffled against the others' original values. For the correlated cardiovascular features HighBP, HighChol, HeartDiseaseorAttack and Stroke, whose mutual Spearman correlations appear in Figure 12 of the Appendix, this splits the shared signal across them, and

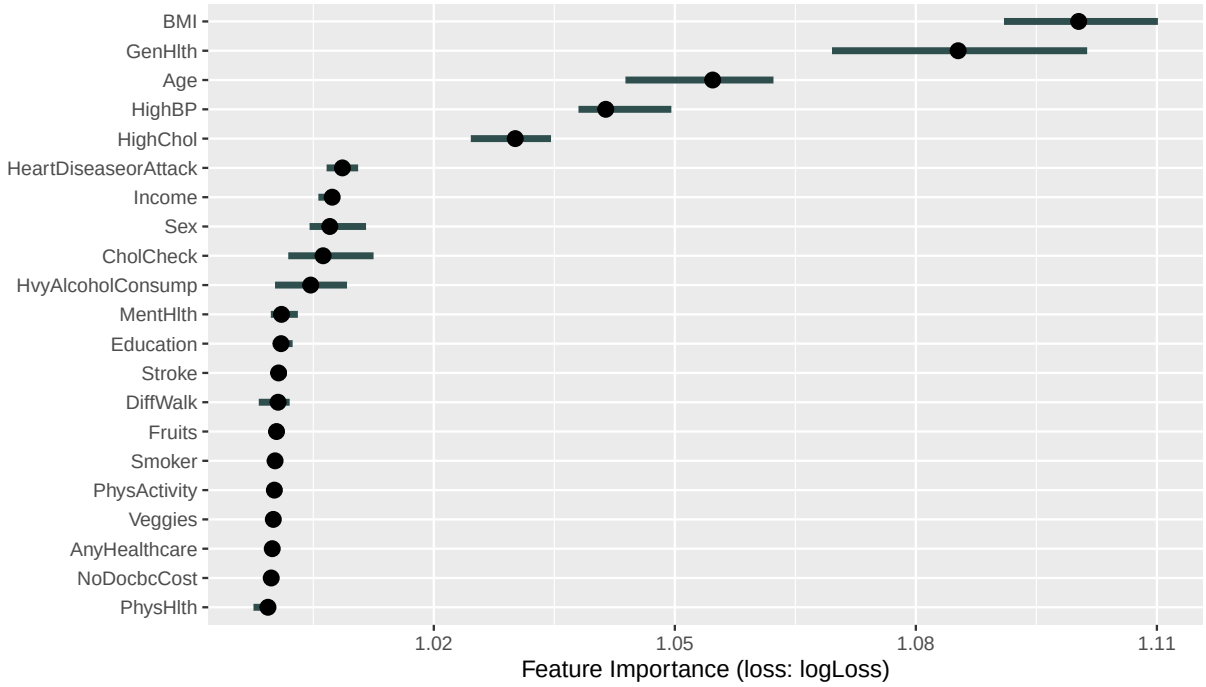


Figure 4: Permutation feature importance on the held-out test subsample (log-loss ratio after shuffling each feature, median and 5-95% band over 10 repetitions). Longer bars mark features the model relies on more.

their separate bars understate the relevance of the group shuffled together. Importance ranks features but says nothing about the shape or direction of their effect, which the next sections cover.

Feature effects: PDP and ICE

Importance ranks features but not the shape of their effect. Partial dependence shows how the predicted probability changes on average as one feature varies. ICE curves show the same for each observation, and the PDP is their average. We plot them together. The ICE curves are parallel when the model is additive in that feature, so any fanning signals an interaction.

On the probability scale the PDP curves in Figure 5 match the GAM smooths. Predicted risk climbs with BMI to about the high-30s before flattening and turning down, rises with Age before levelling off in the oldest brackets, and increases in steps with GenHlth. The ICE curves add more. For BMI and Age they are not parallel. They fan out and sometimes cross, which by Proposition 9.2.2 means the model is not additive in those features. The effect of moving BMI depends on a patient's other characteristics, and the same holds for Age. The GenHlth curves are close to parallel shifts, so its effect is near additive. This fanning is the first sign of interaction in the model and motivates the functional decomposition at the end of the section. It also flags the PDP caveat. The PDP averages the model over the marginal distribution of the other features, so it can place weight on BMI and Age combinations that are sparse or absent in the data, where a tree ensemble extrapolates. We adress this with ALE plots.

Feature effects under correlation: ALE

Partial dependence has a known weakness when features are correlated. It averages the model over feature combinations that never occur, where a tree ensemble extrapolates arbitrarily. ALE plots avoid this by accumulating local changes over the data support.

ALE, shown in Figure 6, recomputes the same effects but accumulates only local, data-supported changes, so it is the estimate to trust when features are correlated. For the piecewise-constant tree ensemble Proposition

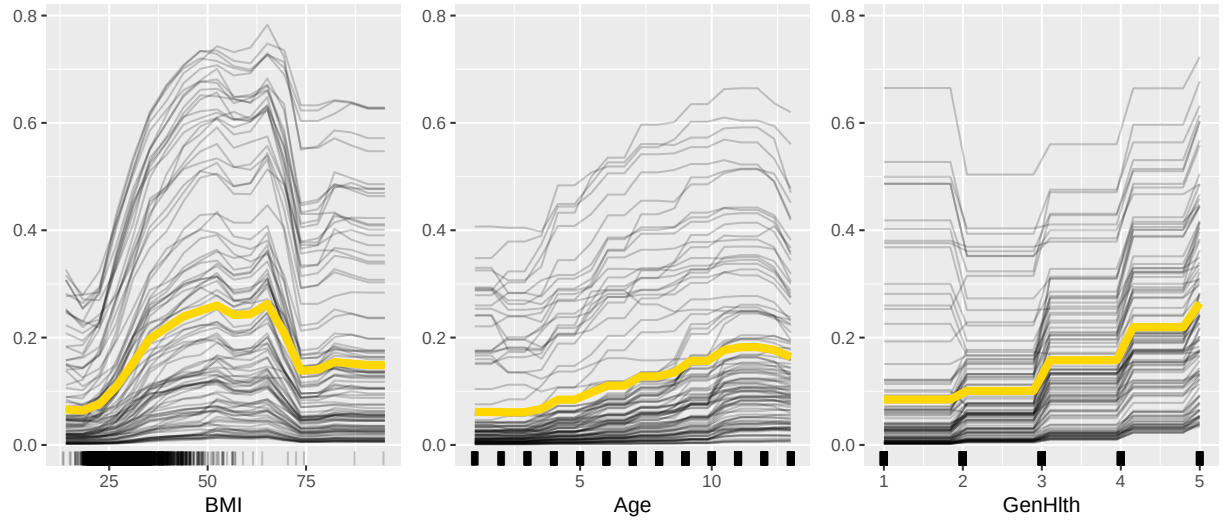


Figure 5: PDP (yellow) and ICE curves (black) for BMI, Age and GenHlth on the probability scale. Non-parallel ICE curves indicate that the feature's effect depends on the other features, i.e. an interaction.

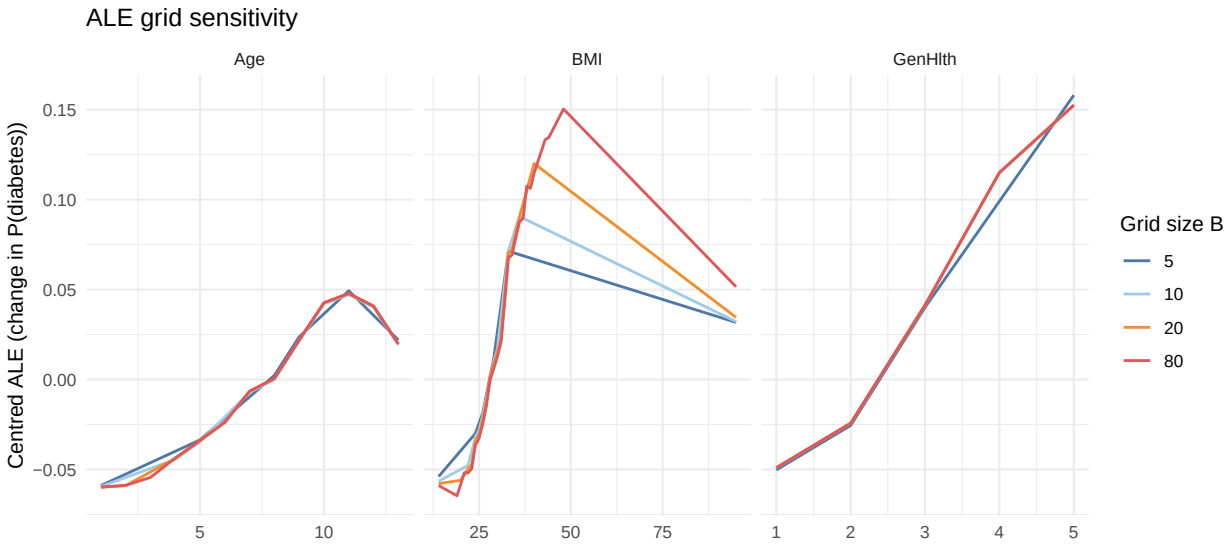


Figure 6: Accumulated Local Effects (ALE) for BMI, Age and GenHlth on the probability scale with varying grid sizes. ALE evaluates the model only on locally data-supported points, so it is the trustworthy effect estimate when features are correlated. The vertical axis is centred, so only differences along each curve are interpretable.

9.4.3 makes the curve a step function whose increments are the left-conditional average jumps. The centred curves agree with the PDP in shape. BMI rises and then bends down past the high-30s, Age increases and levels off, and GenHlth increases throughout. So the extrapolation bias from correlation is mild for these three features. The two methods diverge only in the upper BMI tail. The PDP downturn there is partly extrapolation into a sparse region, while the ALE downturn rests on the few high-BMI patients who occur, so the smaller ALE decline is the more credible one. Even so, neither downturn should be read as a protective effect of severe obesity. The decline falls at the top of the BMI range where very few patients are observed, and it most plausibly reflects the scarcity, selection and noisier self-reported BMI of those extreme cases rather than a genuine fall in risk. The GAM smooth shows the same bend with a sharply widening confidence band over the same region. The defensible reading is that predicted risk rises with BMI and then plateaus, and that the model is not reliable in the sparse high-BMI tail. The vertical axis is centred at zero over each feature's range, so only the differences along a curve are interpretable, not the absolute level. We test different grid sizes to assess the robustness of the ALE estimates, since for piecewise-constant models the ALE curve can depend on the chosen binning. The ALE curves are largely robust to the choice of grid size. For Age, BMI, and GenHlth, the direction and overall shape of the effects remain consistent across grid specifications, indicating that the qualitative conclusions are not driven by the binning choice.

Explanations with SHAP

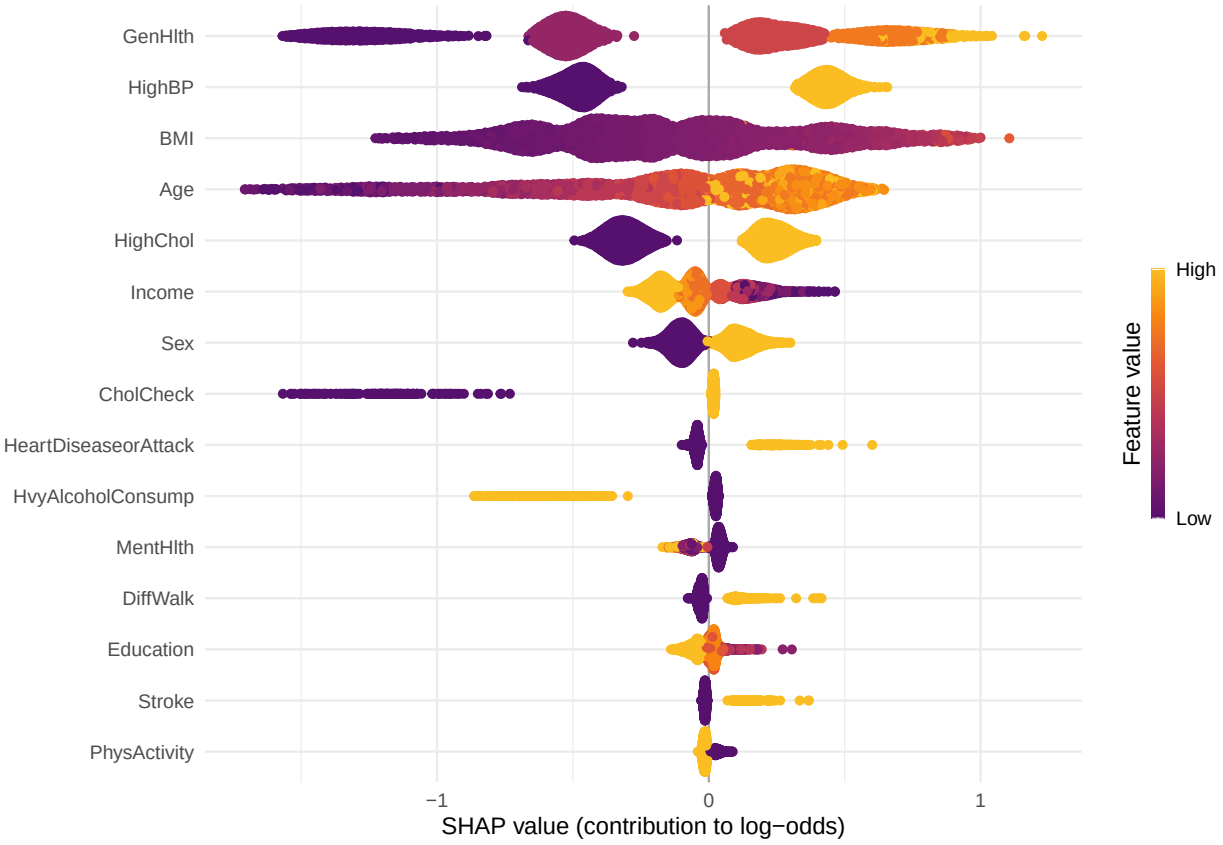


Figure 7: Global TreeSHAP summary on 3,000 sampled patients, computed exactly from the booster with shapviz. Each dot is one patient. Its horizontal position is that feature's TreeSHAP contribution to the predicted log-odds and its colour is the feature value (low to high). Features are ordered top-to-bottom by mean absolute SHAP.

The beeswarm in Figure 7 shows every per-patient SHAP value at once and bridges the importance ranking, which gives only magnitudes, and the individual cases below. Each dot is one of the 3,000 sampled patients,

placed at its contribution to the predicted log-odds and coloured by the feature value. Like the glex components, these are log-odds rather than the probability scale of the PDP, ICE and ALE plots above.

The ordering matches permutation importance, and the colours add the direction it could not show. High values of GenHlth, HighBP, Age and HighChol sit on the positive side and raise predicted risk, while high Income sits on the negative side and lowers it. The two reverse-causality artefacts from the coefficients reappear. CholCheck = 0 produces a long tail of negative contributions and HvyAlcoholConsump = 1 sits to the left. BMI has the widest spread, and its colour is less separated than GenHlth or Age, the footprint of the BMI-Age interaction which is identified further down.

However, we note the problem with TreeSHAP values, that is the algorithm is path-dependent. Because it relies on specific tree splits, two models making identical predictions can yield different SHAP values. While using a background dataset resolves this by recovering exact interventional SHAP values, it introduces its own risk of evaluating unrealistic, out-of-distribution feature combinations.

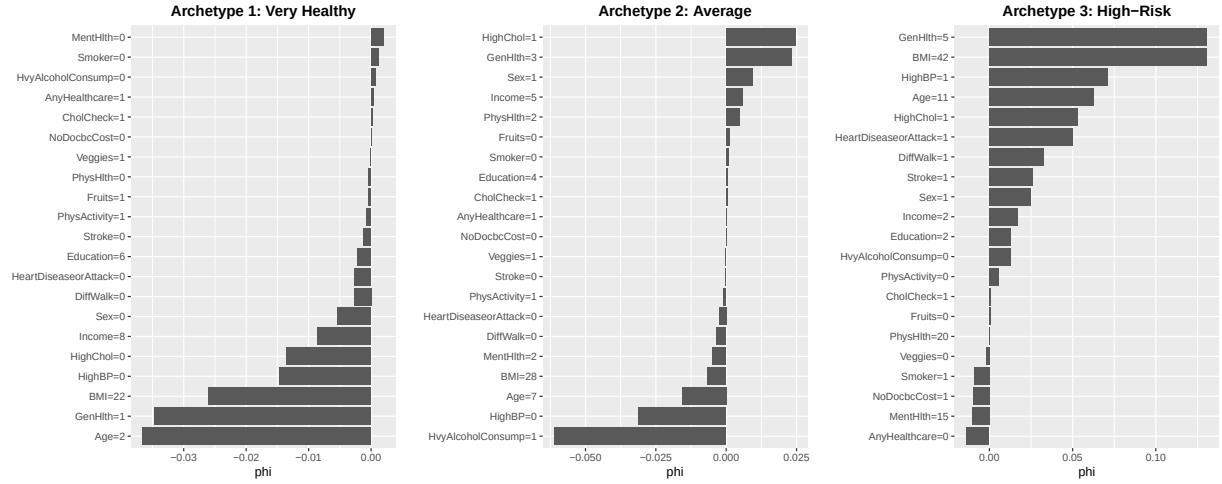


Figure 8: Model agnostic Shapley values (iml, Monte-Carlo sampling with sample.size = 200) for three constructed patients, on the probability scale. Each plot shows the feature contributions to the gap between the patient's predicted probability and the dataset's average prediction. Unlike the TreeSHAP beeswarm above, these are sampled approximations that do not use the booster's tree structure.

In contrast to the beeswarm, these contributions come from iml's model-agnostic Shapley estimator, which samples feature coalitions through the Predictor's probability output rather than reading tree paths, so they are approximate and reported on the probability scale. Each panel of Figure 8 decomposes, for one constructed patient, the gap between that patient's predicted risk and the dataset average, with contributions on the probability scale. For the healthy archetype every bar points left. A low GenHlth, a low BMI and a young Age each pull the prediction below average, and no feature pushes back. The average archetype sits near the baseline. Its contributions are small and offsetting, with HighChol and Sex raising risk and HvyAlcoholConsump = 1 lowering it, the same reverse-causality artefact seen in the coefficient table. The high-risk archetype is the mirror of the first. A high BMI, the worst GenHlth level, high blood pressure, an older age and a cardiovascular history add up to a large positive gap. These are local explanations, but Chapter 10 warns against over-reading a single bar chart. A Shapley value folds a feature's main effect together with its share of every interaction it appears in, so two equal bars can come from a main effect, an interaction, or a mix of both. To separate main effects from interactions we compute the functional decomposition directly.

Functional decomposition with glex

Permutation importance, PDP, ICE, ALE and SHAP each summarise the model along one axis, but none of them shows an interaction on its own. The functional decomposition from Chapter 10 does. It writes any

model as an overall average plus main effects plus interactions of increasing order,

$$\hat{m}_n(x) = m_\emptyset + \sum_k m_{\{k\}}(x_k) + \sum_{j<k} m_{\{j,k\}}(x_j, x_k) + \cdots,$$

where the pairwise term $m_{\{j,k\}}$ is the pure interaction, the part of the joint effect of j and k that is left once we remove what each explains on its own. This is also the object the earlier plots were viewing. A PDP is the intercept plus a main effect, and a Shapley value spreads each component over the features it appears in (Example 10.3.1 and Theorem 10.1.3). SHAP blends main effects and interactions together, while the decomposition pulls them apart.

`glex` computes this decomposition exactly for tree ensembles, since they are piecewise constant. We compute the full decomposition, every interaction order the trees contain, then report the main effects and pairwise interactions in detail before measuring how much the higher-order terms add. `glex` works on the booster's raw log-odds output, so its components are on the log-odds scale, not the probability scale of the iml plots above.

```
set.seed(100)
xgb_model <- at_xgb$learner$base_learner()$model
enc <- po("encode", method = "treatment",
          affect_columns = selector_type(c("factor", "ordered")))
task_test_enc <- enc$train(list(task_test))[[1]]
x_test_enc <- as.data.frame(task_test_enc$data(cols = task_test_enc$feature_names))

glex_results <- cache_rds("glex_decomposition", {
  glex_idx <- sample(nrow(x_test_enc), 500)
  x_glex   <- x_test_enc[glex_idx, ]
  gx <- glex(xgb_model, x_glex) # full decomposition, every order in the trees
  list(gx = gx, x_data = x_glex)
})
gx     <- glex_results$gx
x_glex <- glex_results$x_data
M      <- as.matrix(gx$m) # one column per functional component
ord    <- lengths(strsplit(colnames(M), ":"))
```

Pairwise interactions

We rank the pairwise components by their mean absolute contribution over the sampled patients, which measures how much each interaction moves the predicted log-odds on average.

```
pair_strength <- colMeans(abs(M[, ord == 2, drop = FALSE]))
```

Where Figure 9 ranks these interactions only by magnitude, the surfaces in Figure 10 show their shape. Each tile is the pure pairwise contribution, with both main effects already removed. A pattern is clear across the Age panels, where a risk factor (high cholesterol, high blood pressure, poor self-rated health) adds extra positive log-odds in a younger patient but turns negative in the oldest groups, where that marker is already carried by the main effect. High cholesterol is thus additive at young ages and partly redundant at old ones, and HighBP × GenHlth and Age × GenHlth read the same way. The magnitudes stay modest, from a few hundredths up to about 0.4 on the log-odds scale, so these terms refine the additive fit rather than overturn it. Age × BMI and GenHlth × PhysHlth are diffuse and sparse, consistent with their weaker standing in the ranking.

Higher-order interactions

The pairwise terms are not the whole interaction story. We split the full decomposition by interaction order and report how much absolute effect mass sits at each level.

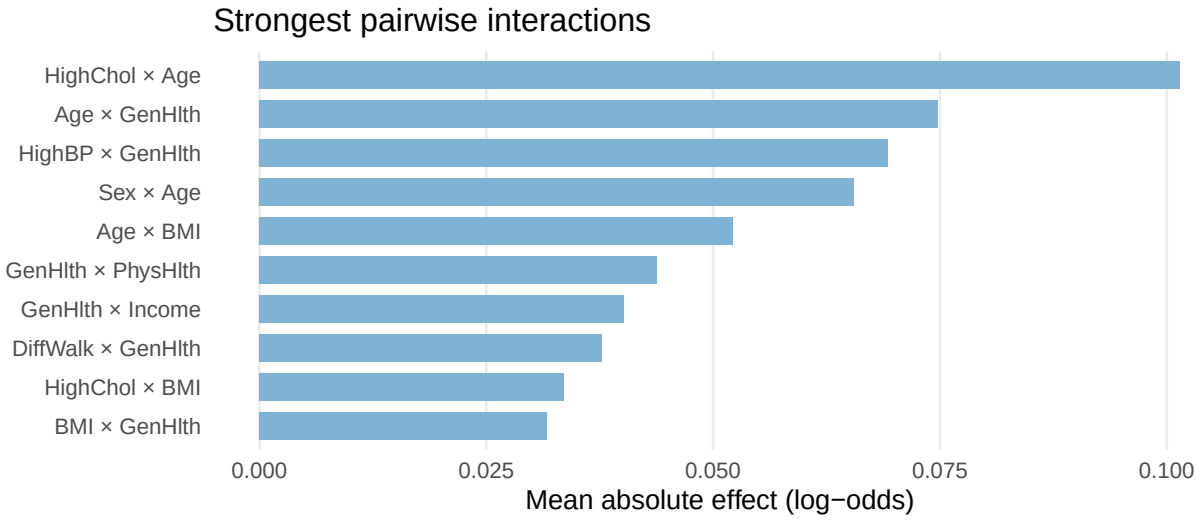


Figure 9: The ten strongest pairwise interactions in the booster, ranked by mean absolute contribution to the predicted log-odds over the 500 sampled patients. Both main effects are removed from each component

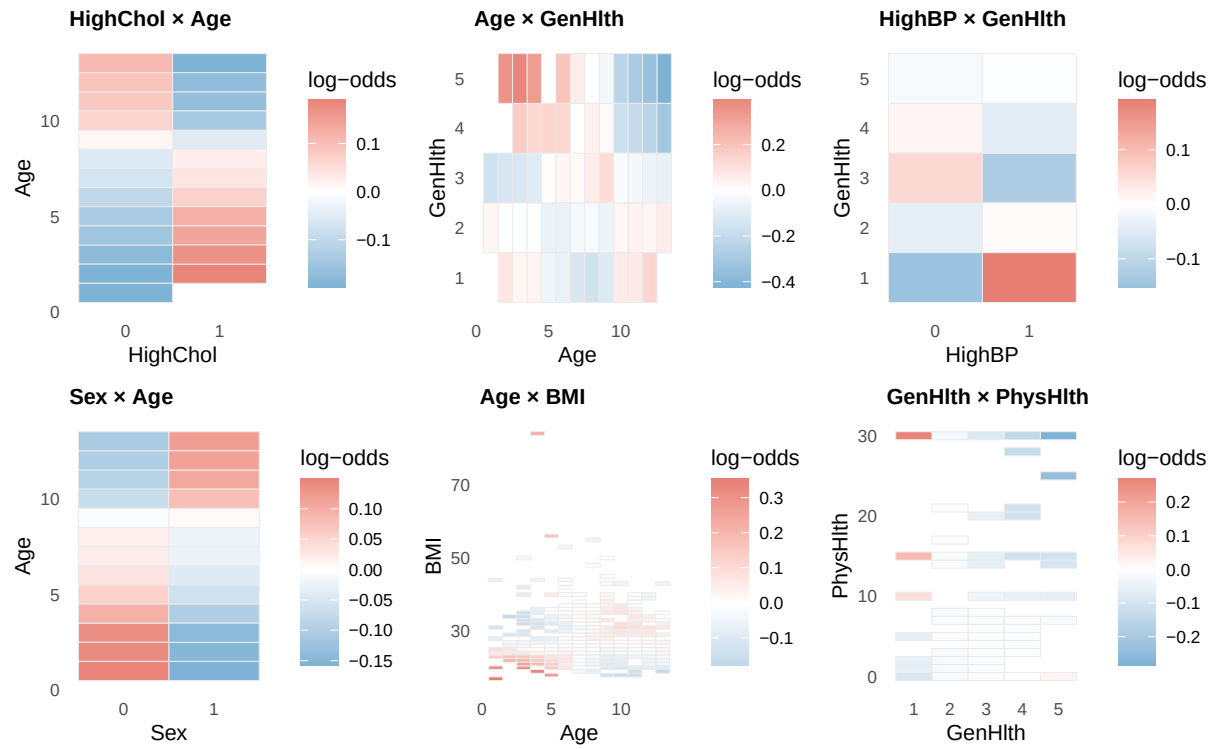


Figure 10: Heatmaps of the top six pairwise interactions from the glex decomposition. Each tile displays the pure interaction effect on the predicted log-odds of diabetes, with both main effects removed. Red areas indicate a positive contribution to diabetes risk (higher log-odds), while blue areas indicate a negative contribution (lower log-odds). Note that sex=1 is male

Table 19: Share of total absolute effect mass by interaction order on the glex sample

Interaction order	Share of effect mass
1 (main effects)	56.2%
2 (pairwise)	26.1%
3	13.1%
4	4.5%
5+	0.0%

As Table 19 shows, main effects carry 56.2% of the absolute effect mass and pairwise terms a further 26.1%, 13.1% at order three, 4.5% at order four, and nothing beyond. A fifth of the mass in higher-order terms is not negligible, so before trusting a low-order reading we have to ask whether it matters for predictions.

Table 20: Higher-order interaction diagnostics for the `glex` decomposition. Each component is $m_T(x_T)$ with interaction order $|T|$. The higher-order remainder is $r_{>2}(x) = \sum_{|T|>2} m_T(x_T)$ and the centered prediction is $\tilde{m}(x) = \sum_T m_T(x_T)$ (log-odds).

Diagnostic	Definition	Value
Absolute mass share	$\sum_{\lvert T\rvert>2} \lvert m_T\rvert \,/\, \sum_T \lvert m_T\rvert$	17.7%
Mean signed shift	$\overline{\lvert r_{>2}\rvert} \,/\, \overline{\lvert \tilde{m}\rvert}$	4.8%
Variance share	$\mathrm{Var}(r_{>2}) \,/\, \mathrm{Var}(\tilde{m})$	0.29%

Table 21: Top 5 Higher order Interaction Effects

Interaction Term	Mean Absolute Value
Age:BMI:GenHlth	0.0198
HighBP:Age:GenHlth	0.0176
HighBP:HighChol:GenHlth	0.0158
HighBP:HighChol:BMI	0.0156
HighChol:Age:GenHlth	0.0139

The three diagnostics reported in table 20 back the low-order reading. The absolute-mass share, 17.7% of total component mass, sums absolute values over the many small order-three-and-above components, so it avoids cancellation and is the conservative measure. The other two let those components aggregate, and most of that mass disappears. Summed with sign, the higher-order remainder $r_{>2}(x) = \sum_{|T|>2} m_T(x_T)$ shifts the prediction by only 4.8% of the centered log-odds on average, and its variance is just 0.29% of the total prediction variance, so it barely separates patients. The few non-negligible higher order terms in Table 21, Age × BMI × GenHlth, HighBP × Age × GenHlth and HighBP × HighChol × GenHlth, are again built from the leading risk factors. Several contain binary HighBP or HighChol and could be shown as conditional two-dimensional surfaces, but at that share of the variance their predictive weight does not justify it.

Interpretability summary

Although XGBoost is often viewed as a black-box model, the combination of permutation importance, PDP/ICE, ALE, SHAP, and the glex functional decomposition provides a remarkably detailed picture of its behaviour. The different methods consistently identify the same key risk factors and reveal how they influence predictions, both globally and for individual patients. Moreover, the decomposition shows that most predictive signal is carried by main effects and pairwise interactions, with higher-order interactions contributing little to prediction variance. Overall, the model is not directly interpretable in the way a linear model is, but its predictions can nevertheless be explained and understood to a substantial degree.

Sensitive and Protected Features

Sex is a demographic attribute that should not, on its own, drive an individual's predicted diabetes risk. Age is a protected characteristic in many jurisdictions, yet it is also among the strongest and most clinically legitimate predictors of diabetes. BMI is directly predictive and clinically central, but it is correlated with socioeconomic status and carries stigma. These three are not sensitive in the same way, and the causal framework of Chapter 11 lets us treat them differently rather than discarding them uniformly. We designate Sex as the protected feature A and treat the remaining variables, Age and BMI included, as part of the permitted features X_J. The distinction we draw is between dependence we judge legitimate and dependence we do not. Age and BMI influence diabetes risk through direct pathways, age through cumulative risk and BMI through insulin resistance, that a useful model should retain. Sex carries no comparable pathway that a clinical screen should use directly. Whatever its predictive value, we do not want an individual's risk to turn on it. By Definition 11.1.5, the controlled direct effect of a permitted feature, obtained by changing that feature while holding the protected feature fixed, is exactly the dependence we keep, while dependence that runs through the protected feature is what we remove. Throughout, we encode Sex as 0 for female and 1 for male, and relabel only for plotting.

Why removing Sex from the inputs is not sufficient

Simply removing the 'Sex' column eliminates only its direct influence, failing to address its indirect effects. Because permitted features often correlate with sex, they continue to carry its signal. For example, if men smoke at significantly higher rates, smoking status can act as a proxy for sex. A model retrained without the explicit 'Sex' variable is free to reconstruct this demographic association through the remaining features. Ultimately, dropping the protected column is not a solution in itself. The direct-effect criterion of Chapter 11 removes the indirect channel as well. It allows a permitted feature to influence the prediction through the causal response obtained when Sex is held fixed, but not through the conditional distribution of Sex among individuals who share the same permitted features.

What the direct-effect reading assumes

The construction is causal only under the no-unobserved-confounding assumption of Definition 11.5.1, that the response under a joint intervention on the permitted and protected features is recovered by the conditional mean, $m(x_J, a) = \mathbb{E}[Y \mid X_J = x_J, A = a]$. This needs the permitted features to be causally upstream of the outcome and no unobserved common cause of either with diabetes. Both are doubtful here. The survey omits obvious confounders such as diet quality, genetic risk and access to care, and several permitted features are plausibly affected by diabetes rather than the reverse. We already read CholCheck and HvyAlcoholConsump as reverse-causality artefacts, and the same concern applies to the self-reported GenHlth, DiffWalk and PhysActivity. Chapter 11 shows that when the assumption fails, averaging the conditional response over A does not return the direct-effect target but leaves a confounding-bias term $\int \{r(x_J, a) - \mu(x_J, a)\}\, d\rho_A(a)$ that fixed weights do not remove.

We therefore read f_{P_A} as the fitted model averaged over Sex with marginal weights, rather than a causal direct effect. The fair prediction is still invariant to flipping an individual's Sex and still costs essentially nothing in log-loss, and these are properties of the fitted function that hold whatever the causal assumption is.

A direct-effect fair model

We use the direct-effect fair averaging rule of Definition 11.5.1. Writing $m(x_J, a)$ for the response under a joint intervention on the permitted and protected features, which under the no-unobserved-confounding assumption of that section is identified by $\mathbb{E}[Y \mid X_J = x_J, A = a]$, the rule is

$$f_\rho(x_J) = \int m(x_J, a)\, d\rho_A(a).$$

We take the marginal reference $\rho_A = P_A$, estimated on the training split and held fixed for every test individual, which is the default discussed in the chapter and which makes f_{P_A} the structural partial dependence of the

permitted features averaged over Sex. For binary Y the response $m(x_J, a)$ is the predicted probability, so the rule averages the fitted model's predicted risk over the two Sex levels with fixed marginal weights. Because the weights do not depend on the individual, the rule uses neither the individual's own Sex value nor the conditional distribution $P_{A|X_J=x_J}$.

We compare three predictors on the held-out test set. The final tuned XGBoost model that includes Sex, the same model class retrained with Sex removed from the inputs, and the direct-effect fair model. All three are derived from or trained in the same way as the final model, so the comparison isolates the role of Sex rather than a change of learner. The fair model reuses the exact tuned booster. The drop-Sex model is retrained with the identical search space, resampling protocol, and tuning budget.

Table 22: Held-out test-set comparison of the final model, the model with Sex removed from the inputs, and the direct-effect fair model. Log-loss is computed identically for all three. The mean risk gap is the difference in average predicted diabetes probability between the male and female groups. It reflects both any use of Sex and any genuine difference in the permitted features across groups.

Model	Log-loss	Mean risk gap (male - female)
Full model (with Sex)	0.3098	0.0205
Drop-Sex model	0.3110	0.0021
Direct-effect fair model	0.3110	-0.0006

Changing only the Sex value of each test individual, while holding all other features fixed, changes the predicted diabetes probability by 0.0218 on average. The drop-Sex model and the direct-effect fair model are by construction invariant to this flip, since the first does not see Sex and the second uses fixed weights. Table 22 shows that all three models attain very similar log-loss, so removing the direct and indirect use of Sex costs essentially nothing in predictive accuracy on this dataset. This is consistent with the earlier finding that Sex is a comparatively weak predictor here, with a logistic log-odds coefficient of only about 0.26, in contrast to the constructed salary example of the exercises in which the protected feature is a strong driver of the permitted feature and of the response.

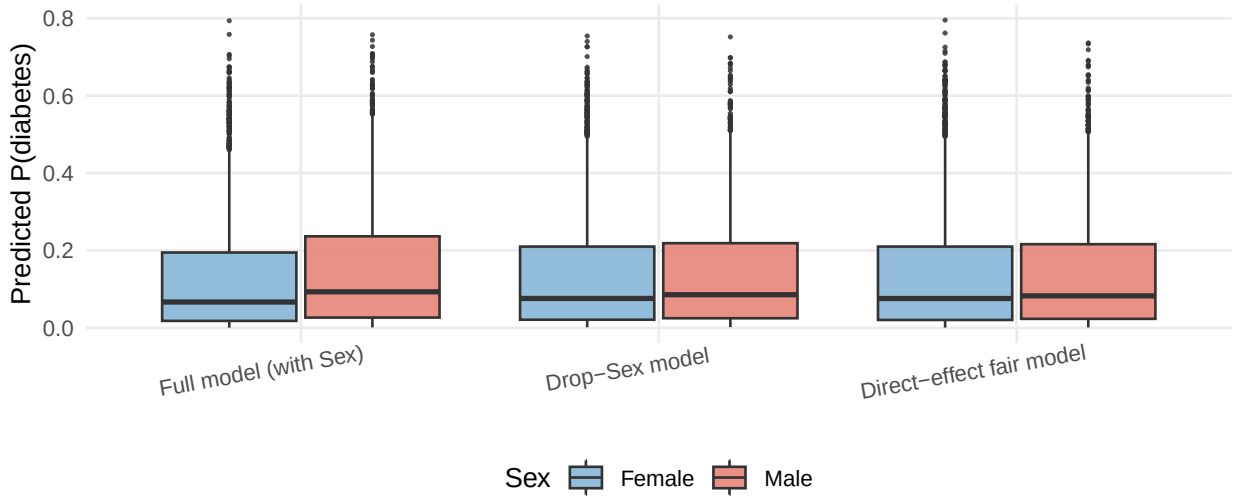


Figure 11: Distribution of predicted diabetes probability on a fixed 5,000-row test subsample, split by Sex, for the three models. For each model the two boxes show the female and male groups. A model that uses Sex shows a visible vertical separation between the two boxes; a model that removes the individual-level effect of Sex shows boxes that differ only to the extent that the permitted features differ across the groups.

The direct-effect fair model assigns the same prediction to two individuals who share the same permitted

features, regardless of their Sex, so it removes Sex at the level of the individual prediction. It does not force the two Sex groups to receive the same average prediction. Any residual gap under the fair model reflects genuine differences in the permitted features across groups, such as differences in the distribution of Age or BMI, whose direct effects the criterion intentionally retains. Equalising group averages is not the direct-effect criterion used here.

Figure 11 shows the predicted-risk distributions split by Sex for the three models, and only the full model separates the female and male boxes vertically, while the two fair predictors differ between groups solely through the permitted features. The direct-effect fair model and the drop-Sex model also differ from each other, by 0.0088 on average across the test set. They differ for the reason given above. The drop-Sex model is retrained, so it can re-route information about Sex through correlated permitted features, whereas the direct-effect fair model removes the Sex channel from the function the final model already learned. That channel has a precise description in the Chapter 10 functional decomposition. By the functional-decomposition remark of Chapter 11, averaging over Sex with marginal weights is equivalent to keeping the intercept together with the components built only from permitted features, and discarding every term in which Sex appears, whether its main effect or any interaction containing it. The averaging form used above is simply the computational route to that same prediction.

Conclusion

Across parametric, non-parametric, tree-based and neural learners the held-out log-loss moved within a narrow band, from the ≈ 0.40 of the constant baseline down to the ≈ 0.31 of the tuned XGBoost. The leaderboard's lesson is that a linear-in-log-odds model already captures most of the signal, and that the remaining gains come almost entirely from the nonlinear effects of BMI and Age together with a handful of pairwise interactions. The GAM recovers nearly all of XGBoost's improvement over the baseline while keeping its partial effects in closed form, so the usual accuracy-interpretability trade-off is here a frontier to move along rather than a hard wall.

We selected XGBoost on log-loss and then reconstructed its logic post hoc. Permutation importance, the PDP, ICE, ALE and SHAP all agree that the model leans on GenHlth, BMI, Age, HighBP and HighChol, the same variables the EDA and the GAM smooths had flagged, and the glex decomposition confirms that the booster's interaction structure runs through Age and GenHlth with negligible mass beyond order three. The two reverse-causality artefacts, CholCheck and HvyAlcoholConsump, recur in every view and are a property of the survey.

Finally, the direct-effect analysis shows that Sex can be removed both directly and indirectly at essentially no cost in log-loss, since it is a weak predictor on this dataset. The residual gap between the male and female groups under the fair model reflects genuine differences in the permitted features, chiefly Age and BMI, whose direct effects the criterion intentionally keeps. For a clinical screening tool, where transparency and an explicit fairness guarantee matter, the GAM or a fair XGBoost would be a defensible deployment, trading a few thousandths of log-loss for a model whose reasoning can be inspected.

Appendix

Figure 12 reports the Spearman correlation matrix of all variables, referenced above when discussing the correlated cardiovascular block.

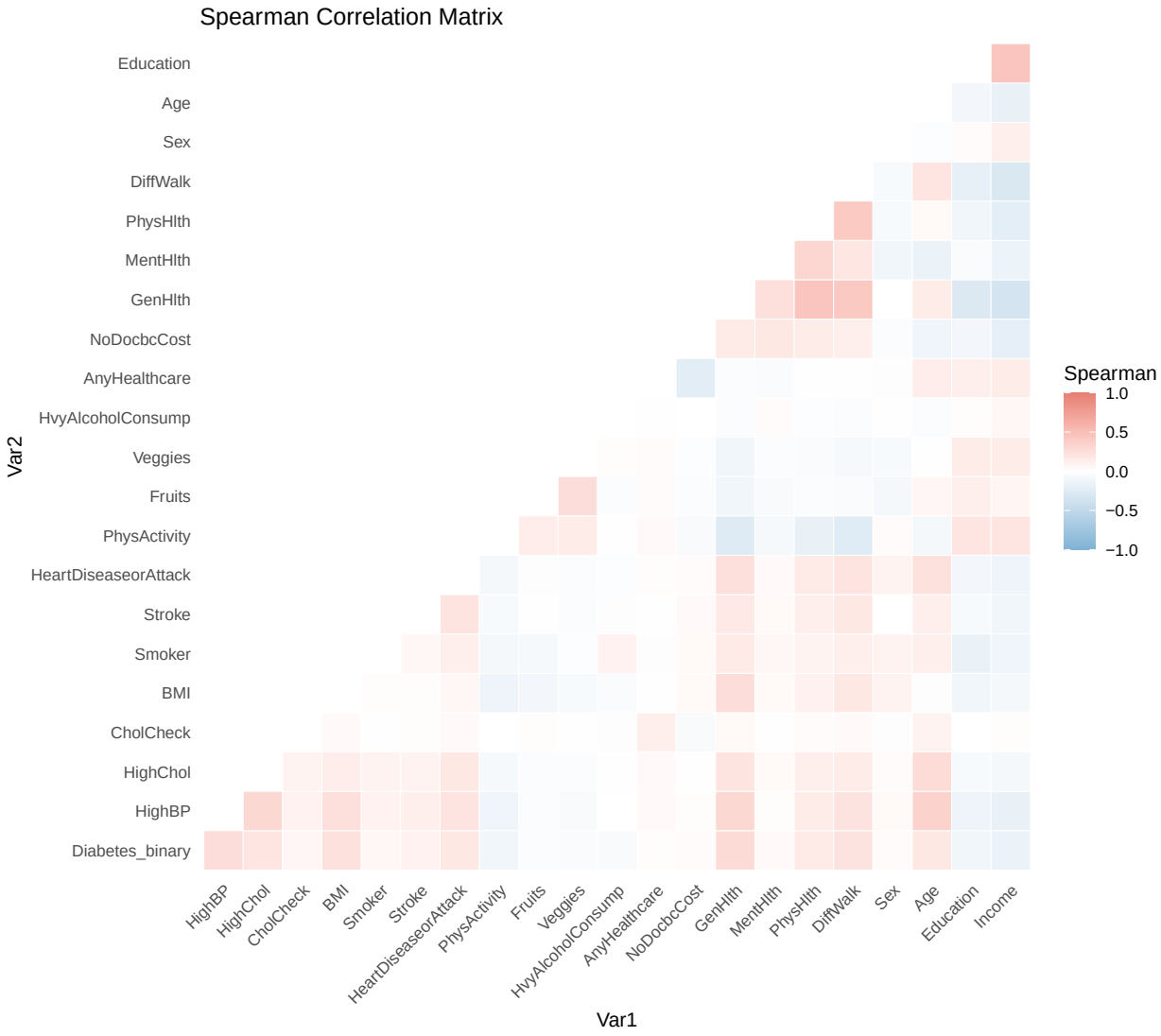


Figure 12: Spearman correlation matrix of all variables